Note technique

1- Architecture

Module	Fichiers	Rôle
Maths	math.hpp	`Vec2` (plan XZ), `Box`, tests de collision disque/AABB, segment contre AABB
Niveau	level.hpp	Murs, bornes, spawns, rayon de l'agent, déplacements
Recherche	navigation.hpp pathfinding.hpp	Graphe pondéré, `Dijkstra`, `AStar`, validation du graphe
NavMesh	navmesh.cpp	Polygones convexes, portails, graphe de régions, localisation, requêtes `FindPath`
Suivi de chemin	navigation_agent.cpp	`Navigator::Plan` (requêtes temporisées) et `Follow` (consommation des waypoints)
Décision	enemy ai.cpp	Etats des ennemis
Combat	gameplay.cpp	Attaque du joueur, dégâts, mort

Débug et Rendu	main.cpp	Affichage du NavMesh, des portails, du chemin sélectionné, HUD, entrées

2. Graphe, coûts et heuristique

Le graphe est une liste d'adjacence (std::vector<std::vector<Edge>>). Un sommet correspond à un polygone convexe du NavMesh. Une arête relie deux polygones qui partagent un portail.

- Coût des arêtes

Pour un portail de milieu m entre les polygones A et B de centres cA et cB :

coût(A, B) = |cA − m| + |m − cB|

Ce coût est positif ou nul, symétrique et correspond à la longueur réelle du trajet que suivra l'agent entre deux centres en passant par le milieu du portail.

- Heuristique de A*

L'heuristique est h(n) = |c(n) − c(but)| (distance euclidienne entre centres de régions).

- Admissible : tout chemin de n au but est une suite de segments dont la somme est au moins la distance en ligne droite entre c(n) et c(but). h ne surestime donc jamais le coût restant.
- Cohérente : pour une arête (a, b), h(a) − h(b) ≤ |ca − cb| ≤ |ca − m| + |m − cb| = coût(a, b). Le premier terme vient de l'inégalité triangulaire sur les centres, le second du passage par m.
- h(but) = 0, ce que AStar vérifie.

Avec une heuristique cohérente, un sommet extrait de la file n'est jamais amélioré ensuite : A* retrouve le même coût optimal que Dijkstra en développant moins de sommets.

3. NavMesh

- Build

Validation de chaque polygone source : au moins trois sommets, coordonnées finies, aucun sommet répété.

Calcul du centre de chaque polygone.

Portails : pour chaque paire de polygones, recherche d'une arête commune.

Graphe : vérification de chaque portail (indices valides, longueur non nulle, extrémités appartenant aux deux polygones).

4. Etats des ennemis

- **Idle :** L'état de base de l'ennemi. Si l'ennemi est un support, alors il va détecter l'allié le plus proche ayant subi des dégâts. S'il est en dehors de la portée de régénération, elle va se déplacer vers celui-ci Pour les autres ennemis, s'il ne repère aucun joueur à proximité, il ne fait rien. Sinon, il passe en Chase.

- **Chase :** Si le joueur est en dehors de la portée de détection, alors l'ennemi repasse en Idle et oublie le chemin. Si le joueur se trouve dans la portée d'attaque de celui-ci, il oublie le chemin et passer en Windup. Sinon, il poursuit le joueur.

- **Windup :** Si la distance entre l'ennemi et le joueur est supérieure à la portée d'attaque de l'ennemi, alors l'ennemi repasse en chase. Sinon, il attaque le joueur puis passe en Recover.

- **Recover :** Suite à l'attaque, l'ennemi ne bouge plus le temps de reprendre son souffle. Il repasse ensuite en Chase.

- **Healing :** Si l'ennemi support détecte un autre ennemi ayant perdu des points de vie, il va alors ajouter 2 points de vie à celui-ci et repasser en Idle.

- **Dead :** Si l'ennemi n'a plus de points de vie, alors il ne fait plus rien et disparaît.

5. Gameplay

Gère surtout le système d’attaque du joueur et la suppression des ennemis à leur mort.

Répartition des tâches :

Harentsoa : navmesh.cpp, gameplay.cpp, enemy_ai.cpp, main.cpp

Maxime : pathfinding.cpp, navigation_agent.cpp, enemy_ai.cpp, main.cpp, level.cpp